

REPORTE FASE 1



García Torres Daniela Catalina
Espinoza Robles Fátima Gabriela
Recamier Acevedo José de Jesús
Lizaola Hernández Aylin Danae
Lomelí Mejía María del Sagrario
Ferrer Saucedo Valeria Lizeth

ARQUITECTURA DE COMPUTADORAS

1. Elementos y características generales del procesador MIPS de 32 bits

La arquitectura MIPS (Microprocessor without Interlocked Pipeline Stages) es uno de los ejemplos más representativos de la filosofía de diseño RISC (Reduced Instruction Set Computer). Su diseño se centra en la simplicidad, la eficiencia y el alto rendimiento mediante la ejecución rápida de instrucciones sencillas.

Las características principales de un procesador MIPS de 32 bits incluyen:

- **Arquitectura de 32 bits:** Los registros, el bus de datos y las direcciones de memoria tienen un tamaño de 32 bits (1 palabra).
- **Formato de instrucción fijo:** Todas las instrucciones tienen una longitud estricta de 32 bits. Esto simplifica enormemente el proceso de decodificación en la Unidad de Control.
- **Arquitectura Load/Store:** Las operaciones aritméticas y lógicas solo se realizan entre registros. El acceso a la memoria principal está estrictamente limitado a las instrucciones de carga (*Load*) y almacenamiento (*Store*).
- **Banco de Registros:** Cuenta con 32 registros de propósito general, lo que permite mantener las variables de uso frecuente dentro del procesador para evitar accesos lentos a la memoria.

2. Set de instrucciones MIPS y definición del ISA para el diseño

El conjunto de instrucciones (ISA) de MIPS se divide en tres formatos principales, dependiendo de cómo se distribuyen los 32 bits de la instrucción:

1. **Tipo R (Registro):** Utilizan tres registros (dos de origen y uno de destino).
2. **Tipo I (Inmediato):** Utilizan dos registros y un valor constante (inmediato) de 16 bits.
3. **Tipo J (Salto):** Utilizan un código de operación y una dirección de salto de 26 bits.

Set de instrucciones específico del proyecto: Para este diseño, se ha definido un ISA reducido basado en las especificaciones del proyecto. Las instrucciones que el procesador será capaz de ejecutar son:

- **Aritméticas y Lógicas (Tipo R):** Add, Sub, Mul, Div, Or, And, slt, nop.
- **Aritméticas y Lógicas (Tipo I):** Addi, Subi, Ori, Andi, Slti.
- **Acceso a Memoria (Tipo I):** Lw, Sw.
- **Control de Flujo (Tipo I y J):** beq, bne, bgtz (Tipo I) y j (Tipo J).

3. Desarrollo de los elementos implementados (Fase 1)

En esta primera fase del proyecto, el objetivo principal fue la implementación del *single datapath* (ruta de datos de un solo ciclo) de MIPS de 32 bits, enfocado exclusivamente en la ejecución de instrucciones de **Tipo R**.

Para lograr esto, se desarrollaron e interconectaron los siguientes módulos de hardware (basados en la Figura 1 y 2 del documento) :

- **Program Counter (PC):** Registro que almacena la dirección de la instrucción actual.
- **Memoria de Instrucciones:** Módulo de solo lectura donde se precargan las instrucciones en código máquina.
- **Banco de Registros (Registers):** Memoria interna que contiene los 32 registros. Para las instrucciones Tipo R, lee dos registros fuentes (**Read register 1** y **Read register 2**) y escribe en un registro destino (**Write register**).
- **ALU (Arithmetic Logic Unit):** Encargada de realizar las operaciones matemáticas (suma, resta) y lógicas (AND, OR) requeridas por la instrucción.
- **Unidades de Control (Control y ALU Control):** La unidad de control principal decodifica los 6 bits más significativos (Opcode). Para las instrucciones Tipo R, delega la decisión final al *ALU Control*, el cual lee los últimos 6 bits de la instrucción (campo *funct*) para determinar la operación exacta que la ALU debe realizar.

El flujo de ejecución de una instrucción Tipo R implementado es el siguiente: El PC indica la dirección a la Memoria de Instrucciones; la instrucción es extraída; los campos de los registros fuente pasan al Banco de Registros; los datos leídos pasan a la ALU, la cual es configurada por el ALU Control; finalmente, el resultado de la ALU se retroalimenta al Banco de Registros para ser guardado.

4. Opciones de algoritmos en ensamblador posibles con este ISA

Respondiendo a la interrogante sobre qué algoritmos se pueden implementar con el conjunto de instrucciones definido en las Tablas 1 y 2, las opciones son bastante amplias. Dado que el ISA incluye operaciones aritméticas completas, acceso a memoria y ramificaciones condicionales (saltos), este procesador es *Turing completo* en la práctica (limitado solo por el tamaño de la memoria).

Algunas opciones de algoritmos viables que podrían proponer para su proyecto son:

1. Cálculo de la Serie de Fibonacci:

- **Por qué es posible:** Requiere sumas (**Add**), inicialización de variables (**Addi**), y un ciclo para iterar n veces usando ramificaciones (**bne** o **bgtz**).

2. Factorial de un número (Iterativo):

- **Por qué es posible:** Se requiere multiplicar el resultado acumulado (**Mul**), restar 1 al contador en cada ciclo (**Subi**) y verificar si el contador ha llegado a cero (**bgtz**).

3. Búsqueda en un Arreglo (Búsqueda Lineal):

- **Por qué es posible:** Implica iterar sobre datos almacenados en memoria. Se usaría **Lw** para traer los datos del arreglo, **beq** o **bne** para comparar si se encontró el número buscado, y **Addi** para avanzar el puntero de memoria.

4. Ordenamiento Burbuja (Bubble Sort):

- **Por qué es posible:** Es uno de los algoritmos más completos para probar un procesador. Requiere manipular un arreglo usando **Lw** y **Sw**, comparar elementos adyacentes usando Set Less Than (**slt**), y realizar ciclos anidados manejados por **bne** y variables temporales.

5. Multiplicación de Matrices o Vectores:

- **Por qué es posible:** Usando **Lw**, operaciones matemáticas acumulativas (**Mul**, **Add**) y ciclos de control.